

- 
- 
- 
- 
- 
- 
- 
- 
- 
- 
- 
- 

会社の PC ですが、active directory を使用して PC にログインを行っています。 どのサーバーに active directory の情報を見に行っているのかがわかりません。どうすればわかりますか。

はい。Windows PC が Active Directory (AD) でログインしている場合、「どのドメインコントローラー (DC)」に認証しに行っているかは、PC 側から確認できます。  
代表的な確認方法を順番に書きます。

---

### 一番簡単な方法 (おすすめ)

コマンドプロンプトを開いて、以下を実行します。

```
echo %LOGONSERVER%
```

表示例：

```
¥¥ADSERVER01
```

これが、現在ログオン時に使われたドメインコントローラーです。

---

### 詳細情報を確認する方法

方法 1：nltest

nltest /dsgetdc:ドメイン名

例：

```
nltest /dsgetdc:example.local
```

結果例：

```
DC: ¥¥ADSERVER01.example.local
```

Address: ¥10.0.0.10

Dom Guid: xxxxx

Dom Name: example.local

どの DC が使われているか、IP アドレスまで確認できます。

---

## 方法 2 : set l

コマンドプロンプトで :

set l

LOGONSERVER= を探します。

---

## PowerShell で確認

\$env:LOGONSERVER

または :

nltest /dsgetdc:\$env:USERDNSDOMAIN

---

## 「いつもどの AD サーバーを見るのか」を確認したい場合

AD は通常、複数のドメインコントローラーが存在し、以下で自動選択されます。

- サイト (拠点)
- DNS
- 応答速度
- 可用性

なので、毎回同じとは限りません。

現在利用可能な DC 一覧は :

nltest /dclist:ドメイン名

例 :

nltest /dclist:example.local

---

## GUI で確認する方法

方法 : システム情報

Win + R → cmd

systeminfo

結果の中の :

ログオン サーバー :

を確認。

---

## DNS から確認したい場合

AD の DC は DNS SRV レコードで配布されています。

nslookup

その後：

set type=SRV

\_ldap.\_tcp.dc.\_msdcs.ドメイン名

例：

\_ldap.\_tcp.dc.\_msdcs.example.local

すると DC 一覧が表示されます。

---

## 会社 PC で注意

会社 PC では：

- nltest が禁止
- PowerShell 制限
- 管理者権限制限

されている場合があります。

その場合でも echo %LOGONSERVER% は大抵使えます。

---

## まず確認すべき順番

1. echo %LOGONSERVER%
2. systeminfo
3. nltest /dsgetdc:ドメイン名

この3つでほぼ確認できます。

echo %LOGONSERVER%、を実行しました結果以下のように表示されました。 %LOGONSERVER%

その結果の場合、環境変数 LOGONSERVER が展開されていません。

よくある原因は次のどれかです。

- コマンドを PowerShell で実行した
- % を全角で入力した
- その PC が AD ログオンしていない
- 環境変数が取得できない特殊状態

まず、どの画面で実行したか確認してください。

---

## 正しい実行方法（重要）

### コマンドプロンプト(cmd.exe)の場合

Win + R → cmd → Enter

その黒い画面で：

```
echo %LOGONSERVER%
```

を実行します。

---

### PowerShell の場合は書き方が違います

もし青い画面（PowerShell）なら：

```
echo $env:LOGONSERVER
```

または：

```
$env:LOGONSERVER
```

です。

PowerShell で %LOGONSERVER% を打つと、そのまま文字列表示になることがあります。

---

## 追加確認

以下も実行してみてください。

### cmd の場合

```
whoami
```

```
systeminfo | findstr /C:"ログオン サーバー"
```

---

### AD 参加しているか確認

```
systeminfo | findstr /C:"ドメイン"
```

または：

```
whoami /fqdn
```

もしワークグループ PC なら、AD ログオンではありません。

---

## 見分け方

### cmd.exe

- 黒画面
- C:¥Users¥xxx>

### PowerShell

- 青っぽい
  - PS C:¥Users¥xxx>
-

まずは、  
\$env:LOGONSERVER  
を試すのが一番早いです。

\$env:LOGONSERVER、を実行しました結果以下のように表示されました。 ¥¥LAN-SERVER5

¥¥LAN-SERVER5 が、現在あなたの PC がログオン認証で利用している Active Directory のドメインコントローラーです。

つまり、

- PC ログイン時
- ユーザー認証
- グループポリシー取得

などで、主に LAN-SERVER5 に問い合わせています。

---

次に確認すると便利なこと

そのサーバーの実体 (IP アドレス)

PowerShell または cmd で：

nslookup LAN-SERVER5

---

ドメイン名確認

\$env:USERDOMAIN

または：

\$env:USERDNSDOMAIN

---

他に存在する AD サーバー一覧

もし nltest が使えるなら：

nltest /dclist:ドメイン名

例：

nltest /dclist:example.local

---

さらに詳しく調べる方法

現在の DC とのセキュアチャネル状態確認：

nltest /sc\_verify:ドメイン名

---

## GUI で見える方法

「Active Directory ユーザーとコンピューター」や「サイトとサービス」が入っていれば、どの DC が存在するか確認できますが、通常はサーバー管理者向けです。

---

## 補足

LOGONSERVER は「ログオン時に利用された DC」を示します。

ただし AD 環境では：

- 複数 DC 構成
- フェイルオーバー
- サイト間切替

があるため、将来的に別サーバーへ切り替わることがあります。

ローカル環境にアプリを作成したいです。 web ブラウザで active directory を使用してログインを行い、ログイン後が成功したら画面に「hello」と表示したいです。

ローカル環境で試すなら、まずは「IIS Express/Kestrel + Windows 認証」で作るのが一番シンプルです。AD に対してパスワードを直接扱わず、ブラウザ/Windows の統合認証に任せる構成にします。

思考時間: 6s

できます。ローカル検証なら **ASP.NET Core + Windows 認証** が一番簡単です。AD の ID/パスワードをアプリで直接扱わず、Windows ログオン済みユーザーをブラウザ経由で認証します。Microsoft 公式でも ASP.NET Core は Windows 認証に対応しており、AddNegotiate() で Negotiate/Kerberos/NTLM 認証を構成できます。

### 1. プロジェクト作成

PowerShell で任意フォルダに移動して：

```
dotnet new web -n AdHelloApp
```

```
cd AdHelloApp
```

```
dotnet add package Microsoft.AspNetCore.Authentication.Negotiate
```

### 2. Program.cs を以下に置き換え

```
using Microsoft.AspNetCore.Authentication.Negotiate;
```

```
using Microsoft.AspNetCore.Authorization;
```

```
var builder = WebApplication.CreateBuilder(args);
```

```
builder.Services
    .AddAuthentication(NegotiateDefaults.AuthenticationScheme)
    .AddNegotiate();
```

```
builder.Services.AddAuthorization(options =>
{
    options.FallbackPolicy = new AuthorizationPolicyBuilder()
        .RequireAuthenticatedUser()
        .Build();
});
```

```
var app = builder.Build();
```

```
app.UseAuthentication();
app.UseAuthorization();
```

```
app.MapGet("/", (HttpContext context) =>
{
    var user = context.User.Identity?.Name ?? "unknown";
    return Results.Content(
        $"""
        <html>
        <body>
        <h1>hello</h1>
        <p>ログインユーザー: {user}</p>
        </body>
        </html>
        """,
        "text/html"
    );
});
```

```
app.Run();
```

### 3. 起動

```
dotnet run
```

表示された URL、例：

`http://localhost:5000`

をブラウザで開きます。

成功すると：

hello

ログインユーザー：ドメイン名¥ユーザー名

のように表示されます。

### 注意点

Chrome / Edge なら社内ドメイン参加 PC では自動ログインされることが多いです。資格情報を聞かれる場合は、URL を localhost ではなく PC 名で開く、またはブラウザの「統合 Windows 認証」の設定が必要になることがあります。

本番運用する場合は IIS 側で Windows 認証を有効化し、匿名認証を無効化する構成が一般的です。Microsoft 公式でも IIS で Windows Authentication を有効化する説明があります。

`dotnet new web -n AdHelloApp`、の意味を教えてください。

`dotnet new web -n AdHelloApp` は、.NET の Web アプリのひな形（テンプレート）を作成するコマンドです。

コマンドを分解すると：

`dotnet new web -n AdHelloApp`

---

dotnet

.NET SDK のコマンドです。

Java でいう：

java

や

mvn

のようなものです。

.NET アプリの：

- 作成
- ビルド
- 実行
- テスト



などを行います。

---

new

「新しいプロジェクトを作る」という意味です。

---

web

作成するプロジェクトの種類（テンプレート）です。

つまり：

Web アプリの最小構成を作る

という意味です。

これは ASP.NET Core の最小 Web アプリになります。

作成されるもの：

- Web サーバー
- HTTP 受付
- ルーティング
- Program.cs

など。

---

-n

name の省略です。

---

AdHelloApp

プロジェクト名です。

つまり：

AdHelloApp という名前の Web アプリを作る

という意味になります。

---

**実際に何が作られるか**

実行すると：

AdHelloApp/

├─ Program.cs

├─ AdHelloApp.csproj

├─ appsettings.json

└─ Properties/

などが作られます。

---

## 特に重要なファイル

Program.cs

Web アプリの開始地点です。

今回あなたが編集するメインファイルです。

---

AdHelloApp.csproj

プロジェクト設定ファイル。

NuGet ライブラリなどを管理します。

---

## つまりこのコマンドを日本語にすると

ASP.NET Core の Web アプリを

AdHelloApp という名前で新規作成する

という意味です。

---

## 実行前に必要なもの

.NET SDK が必要です。

確認：

dotnet --version

バージョンが出れば OK です。

もし：

dotnet は認識されません

なら .NET SDK 未インストールです。

---

## インストール先

.NET SDK は：

[.NET 公式ダウンロードページ](#)

から入れられます。

dotnet new web -n AdHelloApp、を実行しました結果です。 dotnet：用語 'dotnet' は、  
コマンドレット、関数、スクリプト ファイル、または操作可能なプログラムの名前として  
認識されません。名前が正しく記述されていることを確認し、パスが含まれている場合  
はそのパスが正しいことを確認してから、再試行してください。 発生場所 行:1 文字:1  
+ dotnet new web -n AdHelloApp + ~~~~~ + CategoryInfo : ObjectNotFound:  
(dotnet:String) [], CommandNotFoundException + FullyQualifiedErrorId :  
CommandNotFoundException

そのエラーは、.NET SDK がインストールされていない、または PATH が通っていない状態です。

まず .NET SDK をインストールすれば解決できます。

---

## やること

### 1. .NET SDK をダウンロード

公式サイト：

[.NET SDK ダウンロード](#)

---

## おすすめ

現在なら：

- .NET 8 SDK (LTS)  
または
- .NET 9 SDK

を選べば OK です。

会社 PC なら安定版の：

.NET 8 SDK (LTS)

がおすすめです。

---

## 注意

インストールするのは：

SDK

です。

Runtime ではありません。

---

### 2. インストール

ダウンロードした .exe を実行して普通に進めます。

---

### 3. PowerShell を開き直す

インストール後は必ず：

- PowerShell  
または
- cmd

を閉じて開き直してください。

---

#### 4. 確認

`dotnet --version`

例：

8.0.204

のように出れば OK。

---

#### 5. 再実行

`dotnet new web -n AdHelloApp`

---

#### もし会社 PC でインストール不可なら

管理者権限やソフト制限で SDK を入れられない場合があります。

その場合は：

- Visual Studio が既に入っているか確認
- IT 管理者へ依頼
- 開発用 PC を使う

必要があります。

---

#### Visual Studio がある場合

Visual Studio が入っているなら、GUI で作る方が簡単です。

手順

1. [Visual Studio](#) を起動
2. 「新しいプロジェクト」
3. 「ASP.NET Core Empty」
4. 名前：AdHelloApp
5. 作成

でも同じものが作れます。



どれがいいですか。

この画面なら、今は **.NET 10.0 (長期サポート)** を選ぶのがおすすめです。

クリックする場所は：

- .NET 10.0 (最新)
- 「長期サポート」

の行です。

---

### 次の画面で重要

クリックすると、

- SDK
- Runtime
- ASP.NET Runtime

などが出ます。

ここで選ぶのは必ず：

SDK

です。

---

### ダウンロードするもの

Windows なら通常は：

x64 SDK

を選びます。

ファイル名イメージ：

dotnet-sdk-10.x.x-win-x64.exe

---

### インストール後

PowerShell を閉じて開き直し：

```
dotnet --version
```

で確認。

その後：

```
dotnet new web -n AdHelloApp
```

を実行してください。

どこをクリックすればいいですか。

左側の「SDK 10.0.203」の表の中の、  
Windows の x64  
をクリックしてください。  
場所としては：

- 左側ブロック
- 「SDK 10.0.203」
- 「Windows」行
- 「x64」リンク

です。

---

### 間違えやすいポイント

右側の：

ASP.NET Core Runtime

ではありません。

必要なのは：

SDK

です。

---

### ダウンロード後

dotnet-sdk-10.0.203-win-x64.exe

のようなファイルが落ちてきます。

それを実行してインストールしてください。

---

### インストール完了後

PowerShell を一度閉じて、開き直してから：

dotnet --version

を実行してください。

バージョンが表示されたら成功です。

---

```
PS C:\Users\t_matsumura\Documents\AI\chatgpt\AdHelloApp> dotnet run
C:\Users\t_matsumura\Documents\AI\chatgpt\AdHelloApp\Properties\launchSettings.j
son からの起動設定を使用中... ビルドしています...
C:\Users\t_matsumura\Documents\AI\chatgpt\AdHelloApp\Program.cs(4,1): error
CS8802: トップレベルのステートメントを持つことができるのは、1 つのコンパイル ユ
ニットのみのみです。 ビルドに失敗しました。ビルド エラーを修正して、もう一度実行して
ください。
```

原因は、Program.cs の中に 元からあるコードを残したまま、追加で貼り付けた 可能性が



高いです。

Program.cs を開いて、**中身を全部削除**して、以下だけにしてください。

```
using Microsoft.AspNetCore.Authentication.Negotiate;
```

```
using Microsoft.AspNetCore.Authorization;
```

```
var builder = WebApplication.CreateBuilder(args);
```

```
builder.Services
```

```
    .AddAuthentication(NegotiateDefaults.AuthenticationScheme)
```

```
    .AddNegotiate();
```

```
builder.Services.AddAuthorization(options =>
```

```
{
```

```
    options.FallbackPolicy = new AuthorizationPolicyBuilder()
```

```
        .RequireAuthenticatedUser()
```

```
        .Build();
```

```
});
```

```
var app = builder.Build();
```

```
app.UseAuthentication();
```

```
app.UseAuthorization();
```

```
app.MapGet("/", (HttpContext context) =>
```

```
{
```

```
    var user = context.User.Identity?.Name ?? "unknown";
```

```
    return Results.Content($$$$
```

```
    <html>
```

```
        <body>
```

```
            <h1>hello</h1>
```

```
            <p>ログインユーザー: {user}</p>
```

```
        </body>
```

```
    </html>
```

```
    $$$, "text/html");
```

```
});
```

```
app.Run();
```

その後、もう一度：

```
dotnet run
```

を実行してください。

まだエラーが出る場合は、Program.cs の中身をそのまま貼ってください。

```
using Microsoft.AspNetCore.Authentication.Negotiate; using
Microsoft.AspNetCore.Authorization; var builder = WebApplication.CreateBuilder(args);
builder.Services .AddAuthentication(NegotiateDefaults.AuthenticationScheme) .AddNegoti
ate(); builder.Services.AddAuthorization(options => { options.FallbackPolicy = new
AuthorizationPolicyBuilder() .RequireAuthenticatedUser() .Build(); }); var app =
builder.Build(); app.UseAuthentication(); app.UseAuthorization(); app.MapGet("/",
(HttpContext context) => { var user = context.User.Identity?.Name ?? "unknown"; return
Results.Content($"<html> <body> <h1>hello</h1> <p>ログインユーザー:
{user}</p> </body> </html> """, "text/html"); }); app.Run();
```

貼ってくれた Program.cs 自体は問題なさそうです。

このエラーは、同じプロジェクト内に **もう 1 つ別の .cs ファイル** があり、そこにもトップレベルコードがある時にします。

まず以下を実行してください。

```
dir *.cs
```

Program.cs 以外に、例えば Program - コピー.cs や test.cs があれば、それが原因です。

不要な .cs ファイルを削除するか、拡張子を変えてください。

例：

```
ren "Program - コピー.cs" "Program - コピー.txt"
```

その後：

```
dotnet run
```

です。

dotnet run、を実行しました結果です。まだ途中ですか？

成功しています。

今は Web アプリが起動して待機中の状態です。

この表示が重要です：

Now listening on: <http://localhost:5217>

---

次にやること

ブラウザで以下を開いてください。

---

### 成功すると

画面に：

hello

ログインユーザー：ドメイン¥ユーザー名  
が表示されます。

例：

hello

ログインユーザー: COMPANY¥t\_matsumura

---

### 補足

現在 PowerShell が起動し続けているのは正常です。

これは：

Web サーバーが動作中

という意味です。

---

### 停止方法

PowerShell 側で：

Ctrl + C

です。

---

### もしブラウザでエラーになる場合

以下を教えてください。

- ブラウザの表示内容
- URL
- 何か認証ダイアログが出るか

そこから次を案内できます。



ブラウザです。

成功しています。

hello が表示され、さらに

DOMAIN¥t\_matsumura

のように出ているので、Windows 認証でログインユーザーを取得できています。

文字化けしている部分は、HTML に文字コード指定がないためです。Program.cs の HTML 部分を少し直してください。

```
return Results.Content($"
```

```
<!DOCTYPE html>
```

```
<html lang="ja">
```

```
<head>
```

```
    <meta charset="utf-8">
```

```
</head>
```

```
<body>
```

```
    <h1>hello</h1>
```

```
    <p>ログインユーザー: {user}</p>
```

```
</body>
```

```
</html>
```

```
""", "text/html; charset=utf-8");
```

保存後、PowerShell で一度 Ctrl + C、その後もう一度：

```
dotnet run
```

で再表示してください。

URL を開くと hello と表示されるのではなく、URL を開くとユーザー名とパスワードの入力を求められて 入力成功したら、hello と表示したいです。

やりたいことは「**Windows 自動ログイン**」ではなく「**フォームログイン**」ですね。

その場合、今回の Windows 認証 は向いていません。

/login 画面を作って、入力されたユーザー名・パスワードを AD に照合する形にします。

まず追加パッケージを入れます。

```
dotnet add package System.DirectoryServices.Protocols
```

次に Program.cs を全部これに置き換えてください。

```

using System.DirectoryServices.Protocols;
using System.Net;
using System.Text;

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapGet("/", () => Results.Redirect("/login"));

app.MapGet("/login", () =>
{
    return Results.Content("""
<!DOCTYPE html>
<html lang="ja">
<head>
    <meta charset="utf-8">
</head>
<body>
    <h1>ログイン</h1>
    <form method="post" action="/login">
        <div>
            <label>ユーザー名</label>
            <input name="username" />
        </div>
        <div>
            <label>パスワード</label>
            <input name="password" type="password" />
        </div>
        <button type="submit">ログイン</button>
    </form>
</body>
</html>
""", "text/html; charset=utf-8");
});

app.MapPost("/login", async (HttpRequest request) =>

```

```

{
    var form = await request.ReadFormAsync();
    var username = form["username"].ToString();
    var password = form["password"].ToString();

    var domain = "DOMAIN";          // ここを実際のドメイン名に変更
    var ldapServer = "LAN-SERVER5"; // 先ほど確認した AD サーバー

    try
    {
        using var connection = new LdapConnection(ldapServer);
        connection.AuthType = AuthType.Negotiate;
        connection.Credential = new NetworkCredential(username, password, domain);
        connection.Bind();

        return Results.Content("""
<!DOCTYPE html>
<html lang="ja">
<head>
    <meta charset="utf-8">
</head>
<body>
    <h1>hello</h1>
</body>
</html>
""", "text/html; charset=utf-8");
    }
    catch
    {
        return Results.Content("""
<!DOCTYPE html>
<html lang="ja">
<head>
    <meta charset="utf-8">
</head>
<body>

```



```
<h1>ログイン失敗</h1>
<a href="/login">戻る</a>
</body>
</html>
""", "text/html; charset=utf-8");
    }
});
```

```
app.Run();
```

変更する場所はこちらです。

```
var domain = "DOMAIN";
```

```
var ldapServer = "LAN-SERVER5";
```

DOMAIN は、あなたの画面に出ていた DOMAIN¥t\_matsumura の DOMAIN 部分です。

そのまま DOMAIN なら変更不要です。

その後：

```
dotnet run
```

ブラウザで：

```
http://localhost:5217
```

を開くと、ユーザー名・パスワード入力画面になります。